AI-Powered Predictive Maintenance for Metro Train Air Compressors

Dataset: MetroPT-3 Dataset — UCI Machine Learning Repository

Official dataset page

PROJECT PLANNING & MANAGEMENT

AI-Powered Predictive Maintenance for Metro Train Air Compressors

Dataset: MetroPT-3 Dataset

Dataset Source: UCI Machine Learning Repository

Dataset File: MetroPT3(AirCompressor).csv

1. Project Proposal

1.1 Overview

This project aims to develop an AI-powered predictive maintenance system for the Air Production Unit (APU) compressor installed on a metro train using the MetroPT-3 dataset.

The dataset contains real-world multivariate time-series data collected from sensors monitoring the compressor's operating conditions. These measurements include pressure, oil temperature, motor current, and digital signals associated with the compressor's operation and air supply system.

The proposed system will analyze historical sensor readings to identify abnormal operating behavior and detect patterns associated with potential compressor failures, particularly air leaks.

Machine learning and anomaly detection techniques will be investigated to support early fault identification, improve maintenance planning, and reduce unexpected equipment downtime.

The project will also explore time-series feature engineering, model evaluation using documented failure periods, and the development of a deployment prototype for predictive maintenance.

The dataset was published through the UCI Machine Learning Repository and contains real operational data collected between February and August 2020.

UCI Machine Learning Repository+1

1.2 Dataset Summary

Attribute	Details
Dataset Name	MetroPT-3 Dataset
File Name	MetroPT3(AirCompressor).csv
Source	UCI Machine Learning Repository
Number of Records	1,516,948
Number of Features	15
Data Type	Multivariate time-series data
Data Collection Period	February–August 2020
Sampling Frequency	Reported at 1 Hz in the UCI description
Missing Values	None reported by UCI
Equipment	Metro train Air Production Unit (APU) compressor
Primary Application	Predictive maintenance and anomaly detection
Target Availability	No ready-made fault-classification target
Failure Information	Separate reports documenting four air-leak periods

Source: UCI Machine Learning Repository, MetroPT-3 Dataset.

UCI Machine Learning Repository+1

Important: The dataset does not provide a ready-made Fault_Diagnosis column with five classes. The project will therefore investigate anomaly detection and construct time-based labels from documented failure events where appropriate.

1.3 Key Dataset Features

The MetroPT-3 dataset includes 15 variables describing the compressor's operation.

A. Pressure Measurements

- TP2 — Compressor pressure.

- TP3 — Pressure generated at the pneumatic panel.
- H1 — Pressure associated with the discharge behavior of the cyclonic separator filter.
- DV_pressure — Pressure drop associated with the air-drying towers.
- Reservoirs — Downstream air reservoir pressure.

B. Temperature and Electrical Measurements

- Oil_temperature — Compressor oil temperature.
- Motor_current — Electrical current drawn by one phase of the compressor motor.

C. Compressor Operating Signals

- COMP — Electrical signal associated with the compressor air intake valve.
- DV_eletric — Electrical signal controlling the compressor outlet valve.
- Towers — Signal indicating which air-drying tower is operating.
- MPG — Signal associated with compressor activation under load.

D. Monitoring and Control Signals

- LPS — Low-pressure signal that activates when pressure drops below the specified threshold.
- Pressure_switch — Signal associated with the discharge of the air-drying towers.
- Oil_level — Signal indicating that the oil level is below the expected level.
- Caudal_impulses — Pulse count associated with the amount of air flowing from the APU to the reservoirs.

E. Time and Identification Fields

- timestamp — Date and time of each observation.
- index — Record identifier.

The sensor definitions and operating interpretations are based on the dataset documentation.

UCI Machine Learning Repository+1

1.4 Project Objectives

The main objectives of this project are to:

1. Develop an end-to-end predictive maintenance workflow using the MetroPT-3 dataset.
2. Analyze compressor sensor readings to understand normal operating behavior.
3. Detect abnormal patterns in pressure, temperature, motor current, and operating signals.
4. Perform time-series feature engineering using rolling statistics, rate-of-change features, and operating-state indicators.
5. Investigate machine learning approaches for anomaly detection and failure prediction.
6. Evaluate the effectiveness of the developed approach using documented failure periods and suitable time-based evaluation methods.
7. Develop a prototype prediction API for integrating the selected model into a maintenance workflow.
8. Establish a reproducible model development, monitoring, and retraining process.

1.5 Project Scope

In Scope

- Dataset collection and validation.
- Data exploration and preprocessing.
- Exploratory Data Analysis (EDA).
- Time-series analysis and visualization.
- Sensor correlation analysis.
- Feature engineering and rolling-window statistics.
- Anomaly detection using suitable machine learning algorithms.
- Failure-event-based evaluation.
- Model comparison and performance evaluation.
- Model saving and reproducibility.
- API development using FastAPI.

- Docker-based deployment prototype.
- Data drift monitoring and retraining planning.

Out of Scope

- Physical installation of sensors.
- Direct connection to a live metro train.
- Real-time deployment on railway infrastructure.
- Guaranteed prediction of every future failure.
- Production-grade railway safety certification.
- Autonomous maintenance decisions without human verification.

1.6 Project Deliverables

No.	Deliverable	Owner
1	Dataset Exploration Report	Malak Hany
2	EDA Notebook	Malak Hany
3	Cleaned and Preprocessed Dataset	Malak Hany
4	Advanced Time-Series Analysis Report	Jana Mohamed
5	Sensor Visualizations and Dashboard	Jana Mohamed
6	Feature Engineering Summary and Feature-Ready Dataset	Jana Mohamed
7	Model Evaluation Report	Habiba Kamal
8	Python Model Training and Evaluation Code	Habiba Kamal
9	Final Model File and Reproducibility Notes	Habiba Kamal
10	Deployed API / Deployment Prototype	Sama Saed
11	MLOps and Deployment Report	Sama Saed
12	Monitoring and Retraining Setup	Sama Saed
13	Final Documentation and Project Demonstration	All Team Members

2. Project Plan and Timeline

2.1 Gantt Chart Overview

The project is planned to run from October 9, 2026, to December 4, 2026.

Phase	Task	Start Date	End Date	Duration
1	Project Planning and Management	9 Oct 2026	16 Oct 2026	1 week
2	Literature Review	9 Oct 2026	16 Oct 2026	1 week
3	Requirements Gathering	9 Oct 2026	16 Oct 2026	1 week
4	System Analysis and Design	9 Oct 2026	6 Nov 2026	4 weeks
5	Implementation (Source Code and Execution)	16 Oct 2026	30 Nov 2026	Approximately 6.5 weeks
6	Testing and Quality Assurance	16 Oct 2026	4 Dec 2026	Approximately 7 weeks
7	Final Presentation and Reports	16 Oct 2026	4 Dec 2026	Approximately 7 weeks

2.2 Weekly Milestones

Week	Milestone	Key Activities	Owner
Week 1	M1: Data Collection, EDA, and Preprocessing	Load the MetroPT-3 dataset, inspect all 15 variables, validate timestamps, investigate data quality, perform initial EDA, and save the prepared dataset.	Malak Hany
Week 2	M2: Advanced Analysis and Feature Engineering	Analyze sensor trends, identify operating patterns, investigate failure periods, engineer rolling statistics and time-series features, and build a dashboard.	Jana Mohamed

Week 3	M3: Anomaly Detection and Model Evaluation	Define the modeling approach, train candidate models, evaluate anomaly scores against documented events, and select the most suitable model.	Habiba Kamal
Week 4	M4: MLOps, Deployment, and Monitoring	Track experiments, save the model, develop a FastAPI endpoint, package the application using Docker, and design monitoring and retraining workflows.	Sama Saed

2.3 Resource Allocation

Resource	**Purpose**
Python	Core programming language
Jupyter Notebook	Data analysis and experimentation
MetroPT3(AirCompressor).csv	Primary dataset
Pandas	Data manipulation
NumPy	Numerical operations
SciPy	Statistical and signal-processing utilities
Scikit-learn	Preprocessing, anomaly detection, and evaluation
Matplotlib and Seaborn	Data visualization
Plotly	Interactive visualizations
Random Forest	Potential supervised baseline if valid labels are constructed
Isolation Forest	Unsupervised anomaly detection
Local Outlier Factor (LOF)	Alternative anomaly detection approach
XGBoost	Potential supervised baseline if reliable event-based labels are available
PyTorch or TensorFlow	Optional deep-learning experiments

MLflow	Experiment tracking
DVC	Data and pipeline versioning
FastAPI	Prediction API
Docker	Application containerization
GitHub	Version control and documentation

3. Task Assignment and Roles

3.1 Team Responsibilities

1. Malak Hany — Data Engineer

Main responsibility: Data Collection, Exploration, and Preprocessing.

Tasks:

- Load the MetroPT-3 CSV file.
- Inspect dataset dimensions, column names, and data types.
- Parse the timestamp column and verify chronological ordering.
- Check duplicate records and timestamp consistency.
- Investigate sensor values and unexpected operating states.
- Perform initial exploratory analysis.
- Validate the dataset's documented absence of missing values against the actual file.
- Investigate sampling intervals and potential gaps.
- Prepare the cleaned dataset and EDA Notebook.
- Document all preprocessing decisions.

Deliverables:

- Dataset Exploration Report.
- EDA Notebook.
- Cleaned and Preprocessed Dataset.

2. Jana Mohamed — Data Analyst / Feature Engineer

Main responsibility: Advanced Time-Series Analysis and Feature Engineering.

Tasks:

- Analyze trends in pressure, oil temperature, and motor current.
- Visualize sensor behavior over time.
- Investigate operating states using digital signals.
- Examine pressure relationships and potential abnormal patterns.
- Analyze the documented air-leak periods.
- Create rolling means, standard deviations, minimums, and maximums.
- Calculate rate-of-change features.
- Develop sensor-based anomaly indicators.
- Build an interactive dashboard using Plotly or Streamlit.
- Prepare the Advanced Analysis Report and Feature Engineering Summary.

Deliverables:

- Advanced Time-Series Analysis Report.
- Visualizations and Dashboard.
- Feature Engineering Summary.
- Feature-Ready Dataset.

3. Habiba Kamal — Machine Learning Engineer

Main responsibility: Anomaly Detection, Model Development, and Evaluation.

Tasks:

- Define the modeling objective based on the available sensor data and failure reports.
- Establish a normal-operation baseline using a suitable training period.
- Select and train candidate anomaly detection algorithms.
- Investigate Isolation Forest and other suitable approaches.
- Explore supervised classification only if reliable event-based labels can be constructed.

- Develop an appropriate time-aware training and testing strategy.
- Evaluate the models using anomaly detection and event-based metrics.
- Investigate false alarms and missed failure events.
- Select the final model based on validation results.
- Save the trained model and prepare reproducibility notes.
- Produce the Model Evaluation Report.

Deliverables:

- Model Training and Evaluation Code.
- Model Evaluation Report.
- Final Trained Model.
- Model Performance Comparison.

4. Sama Saed — MLOps Engineer

Main responsibility: Deployment, Monitoring, and Model Lifecycle Management.

Tasks:

- Implement experiment tracking using MLflow.
- Organize model artifacts and version information.
- Develop a FastAPI endpoint for anomaly scoring.
- Validate incoming sensor data.
- Package the application using Docker.
- Design a monitoring process for sensor distributions and anomaly scores.
- Define configurable alert thresholds.
- Establish a proposed model retraining workflow.
- Test the API and deployment prototype.
- Prepare the MLOps and Deployment Report.

Deliverables:

- Deployed API / Deployment Prototype.

- MLOps and Deployment Report.
- Monitoring and Retraining Setup.
- Final Project Demonstration.

3.2 Responsibility Matrix (RACI)

RACI Definitions

- R — Responsible: Performs the task.
- A — Accountable: Ultimately accountable for task completion.
- C — Consulted: Provides input or technical advice.
- I — Informed: Receives progress updates.

Task	Malak	Jana	Habiba	Sama
Data Collection and Loading	R	C	I	I
EDA and Cleaning	R	C	I	I
Advanced Analysis	C	R	C	I
Feature Engineering	C	R	C	I
Anomaly Detection and Model Development	I	C	R	C
Model Evaluation	I	C	R	C
MLOps and Deployment	I	I	C	R
Documentation	R	R	R	R

4. Risk Assessment and Mitigation Plan

No.	Risk	Likelihood	Impact	Mitigation Strategy
1	Large dataset size and memory limitations	Medium	High	Process the CSV in chunks or use efficient data types and suitable storage formats.

2	Irregular sampling intervals or timestamp gaps	Medium	Medium	Inspect timestamp differences and document gaps before time-series analysis.
3	Unusual sensor readings	Medium	Medium	Investigate abnormal values in their operating context instead of removing them automatically.
4	Lack of ready-made fault labels	High	High	Use anomaly detection and carefully constructed event-based labels from documented failure reports.
5	Class imbalance in event-based labels	High	High	Use suitable evaluation metrics, event-aware validation, and training-only resampling if needed.
6	Time leakage during model development	Medium	High	Use chronological splitting and ensure rolling features do not include future observations.
7	False alarms from anomaly detection	Medium	High	Tune thresholds on validation data and assess false alarms per unit of time.
8	Model overfitting	Medium	Medium	Use a chronological validation period and evaluate on a later, unseen time period.
9	API or deployment issues	Low	Medium	Test the API independently and use Docker for a reproducible deployment environment.
10	Data or model drift after deployment	Medium	High	Monitor sensor distributions and anomaly-score behavior and define retraining criteria.
11	Limited number of documented failures	High	High	Emphasize anomaly detection, event-level evaluation, and limitations in the final report.
12	Team member unavailability	Low	Medium	Maintain shared documentation, version-controlled code, and knowledge sharing.

5. Key Performance Indicators (KPIs)

The project will use the following proposed KPIs to evaluate model quality and deployment readiness.

Because MetroPT-3 does not provide a ready-made fault label for every row, these targets must be interpreted according to the final evaluation method.

KPI	Proposed Target	Measurement Method
Event Recall	≥ 80%	Percentage of documented failure events detected within the defined warning window
Event Precision	≥ 80%	Percentage of generated event alerts associated with a verified event under the evaluation rules
False Alarm Rate	Defined after baseline evaluation	Number of false alerts per operating day or hour
Detection Lead Time	Reported in minutes or hours	Time between the first valid alert and the documented failure onset
Anomaly Detection Performance	Baseline comparison and improvement	Compare anomaly scores against event periods and normal-operation periods
API Response Time	< 500 ms	API testing using a defined request size and environment
System Uptime	≥ 99%, if hosted continuously	Deployment monitoring
Data Drift Detection	Within 24 hours	Monitoring and alerting tests
Retraining Workflow	Successfully tested	Documented retraining and validation procedure
Documentation Completeness	100%	GitHub repository checklist

6. Dataset-Specific Plan of Action

6.1 Week 1 — Data Engineering

Responsible: Malak Hany

Tasks

1. Load MetroPT3(AirCompressor).csv.
2. Inspect the number of records, columns, and data types.
3. Parse timestamp into a datetime format.
4. Sort the dataset chronologically.
5. Check duplicate timestamps and records.
6. Verify the sampling frequency and identify missing time intervals.
7. Check for missing values, even though none are reported by UCI.
8. Examine the distributions and operating ranges of pressure, temperature, and motor-current measurements.
9. Inspect digital signals and their possible operating states.
10. Save the validated dataset and initial EDA Notebook.

Deliverables

- Dataset Exploration Report.
- EDA Notebook.
- Validated and Preprocessed Dataset.

6.2 Week 2 — Advanced Analysis and Feature Engineering

Responsible: Jana Mohamed

Tasks

1. Create time-series visualizations for:

- TP2
- TP3
- H1
- DV_pressure
- Reservoirs
- Oil_temperature

- Motor_current

2. Investigate relationships between pressure, motor current, and compressor operating states.

3. Analyze digital signals, including COMP, DV_eletric, Towers, MPG, and LPS.

4. Examine the documented failure periods to identify possible changes in sensor behavior.

5. Create rolling statistical features, including:

- Rolling mean.
- Rolling standard deviation.
- Rolling minimum and maximum.
- Pressure differences.
- Rate of change in temperature and motor current.

6. Explore the relationship between reservoir pressure and compressor pressure.

7. Develop visualizations comparing normal-operation periods with documented failure periods.

8. Build an interactive dashboard using Plotly or Streamlit.

Deliverables

- Advanced Analysis Report.
- Sensor Visualizations.
- Interactive Dashboard.
- Feature Engineering Summary.
- Feature-Ready Dataset.

6.3 Week 3 — Anomaly Detection and Model Development

Responsible: Habiba Kamal

Tasks

1. Define the modeling objective

The primary objective is to detect abnormal compressor behavior and investigate whether sensor patterns can provide early warning of documented air-leak events.

2. Establish a baseline

- Select an appropriate early training period.
- Investigate normal operating behavior.
- Use the training data to fit preprocessing steps and anomaly detection models.
- Keep later periods for validation and testing.

3. Train candidate models

- Isolation Forest for unsupervised anomaly detection.
- Local Outlier Factor as an alternative baseline where its computational requirements are manageable.
- A supervised model such as Random Forest or XGBoost only if valid event-based labels are constructed.
- Optional deep-learning approaches if justified by the available data and project resources.

4. Construct event-based labels where appropriate

Use the documented failure reports to define event periods and, if needed, pre-failure warning windows.

The event labels should be created using timestamps, not by inventing a fault-classification column that does not exist in the original dataset.

5. Evaluate model performance

- Event Recall.
- Event Precision.
- False alarms per operating hour or day.
- Detection lead time.
- Confusion matrix and F1-Score if reliable binary labels are constructed.
- ROC-AUC where suitable and supported by the labels.

6. Investigate failure cases

Review examples of:

- Detected abnormal periods.
- Missed documented failures.
- False alarms during normal operation.
- Sensor patterns that precede failure events.

7. Select and save the final model

Select the model based on validation results, detection quality, false-alarm behavior, and computational requirements.

Save the model and its preprocessing configuration in a reproducible format.

Deliverables

- Model Training and Evaluation Code.
- Model Evaluation Report.
- Final Trained Model.
- Model Comparison and Error Analysis.

6.4 Week 4 — MLOps and Deployment

Responsible: Sama Saed

Tasks

1. Use MLflow to track experiments, parameters, metrics, and artifacts.
2. Organize dataset versions and preprocessing configurations.
3. Develop a FastAPI endpoint:

POST /predict

4. Design the endpoint to accept sensor measurements or a suitable time window and return an anomaly score, detection result, or warning status.
5. Validate input fields and handle invalid requests.
6. Package the API using Docker.
7. Monitor the distribution of sensor values and model anomaly scores.
8. Define alert thresholds based on validation results.
9. Design a retraining workflow for newly available data.

10. Test the complete inference pipeline.
11. Prepare deployment documentation and the final demonstration.

Deliverables

- Deployment Prototype.
- FastAPI Prediction Endpoint.
- Docker Configuration.
- MLOps and Deployment Report.
- Monitoring and Retraining Plan.

7. Expected Project Outcome

The expected outcome is an AI-powered predictive maintenance prototype capable of analyzing sensor measurements from a metro train's Air Production Unit compressor and identifying potentially abnormal operating behavior.

The system will combine time-series analysis, feature engineering, anomaly detection, failure-event evaluation, and deployment practices.

The final project will demonstrate how real industrial sensor data can be used to investigate abnormal operating conditions and support maintenance planning.

The system's ability to predict failures in advance will be assessed experimentally. The existence of anomaly scores alone will not be considered proof of successful failure prediction; performance must be demonstrated using documented events and an appropriate time-based test procedure.

8. Conclusion

This project proposes an end-to-end predictive maintenance workflow based on the MetroPT-3 dataset. By combining data engineering, time-series analysis, anomaly detection, model evaluation, and MLOps, the team will develop a reproducible prototype for monitoring compressor behavior and investigating early warning signals.

The project will conclude with a validated dataset, an analysis and visualization dashboard, a trained anomaly detection model, an evaluation report, a deployment prototype, and final documentation.

The results will provide an experimental assessment of the usefulness and limitations of machine learning for predictive maintenance in railway compressor systems.

9. Dataset Reference

Davari, N., Veloso, B., Ribeiro, R. P., Pereira, P. M., & Gama, J. (2021). MetroPT-3 Dataset. UCI Machine Learning Repository.

Dataset page: https://archive.ics.uci.edu/dataset/791/metropt+3+dataset

DOI: 10.24432/C5VW3R.

UCI Machine Learning Repository+1